

A Codebase RAG Assistant: RepoMentor

A local, citation-grounded RAG assistant for understanding unfamiliar GitLab codebases.

[Python](#) [GitLab REST API](#) [Sentence Transformers](#) [BM25](#) [NumPy](#) [Ollama](#)

Project summary

RepoMentor lets engineers ask natural-language questions about a repository. It retrieves the most relevant code and documentation, generates an answer locally, and links it to exact files, commits, and line ranges. The goal is faster onboarding without treating the language model as an unverified source of truth.

Problem and approach

Problem

Knowledge is distributed across code, tests, configuration, and documentation. Manual exploration is slow, while a general chatbot may answer without repository evidence.

Approach

Retrieve before generating, combine semantic and exact-term search, run locally, and attach citations so every answer can be checked against the repository.

Demonstration context

Simulated company	The open-source <code>full-stack-fastapi-template</code> was imported into a private GitLab project to represent a small company's codebase.
Why this repository	It contains frontend, backend, authentication, database models, tests, Docker, configuration, and documentation.
Generalizability	Another supported GitLab project can be indexed by changing the project path and read-only access token.

End-to-end pipeline

01 Ingest and filter Download supported files and exclude generated or low-value content.	02 Chunk and preserve metadata Create overlapping chunks with paths, commits, and line ranges.
03 Build the search index Create semantic embeddings and a BM25 index for every chunk.	04 Retrieve and expand Fuse both rankings, then use identifiers for a second search.
05 Extract grounded facts Give each source separately to the local LLM to reduce confusion.	06 Return cited answers Attach file, commit, and line links in Python.

Prototype snapshot 188 files downloaded • 324 searchable chunks • 384-dimensional embeddings • 0 download failures

Engineering design and evaluation

Key design decisions

Decision	Rationale
Hybrid retrieval	Semantic embeddings capture meaning; BM25 captures exact code identifiers and technical terms.
Rank fusion	Reciprocal Rank Fusion combines rankings without requiring raw scores to share one scale.
NumPy before FAISS	Direct search is sufficient for 324 chunks; FAISS becomes useful as the corpus grows.
Source-by-source extraction	Giving the small LLM one source at a time reduces source confusion and unsupported synthesis.
Programmatic citations	The application attaches repository metadata rather than relying on the LLM to reproduce it.

Local models and their roles

Component	Technology	Role
Embedding model	all-MiniLM-L6-v2	Creates 384-dimensional semantic vectors.
Keyword retrieval	BM25	Finds exact code and configuration terms.
Language model	qwen3:1.7b	Extracts facts and writes answers from evidence.
Local runtime	Ollama	Runs the approximately 1.4 GB model locally.

Note: The models were integrated, not trained or fine-tuned. Local inference avoids per-query API cost and keeps repository content on the user's machine.

Evaluation strategy

Evaluation separates retrieval failures from generation failures.

Layer	Question	Measures
Retrieval	Did the system find the correct evidence?	Hit@K, Recall@K, Precision@K, MRR
Generation	Is the answer correct and supported?	Correctness, faithfulness, citation accuracy, relevance, completeness
System	Is the application practical to run?	Latency, memory, indexing time, storage, cost

Current limitations

The CLI prototype requires re-indexing after repository changes, and the small local model can omit or repeat details. Evaluation is still required to measure correctness.

Example question: "How are user passwords handled?"

Expected evidence: password hashing, verification, user creation, password updates, and recovery code.